

Abstract Data Types

L33 - Structures

Department of Computer Science
University of Pretoria

04 June 2024

- Structures can be stored in arrays
- Can be used instead of parallel arrays (e.g. array of month names and array of month days)

```
const int SIZE = 20;  
Student studentList[SIZE];
```

- Individual structures accessible using **subscript notation**
- Fields within structures accessible using **dot notation**:

```
cout << studentList[5].studentNum;
```

Code Example

```
1  struct Student {  
2      string studentNum;  
3      string firstName;  
4      string surname;  
5      int marks;  
6  };  
7  
8  int main() {  
9      const int SIZE = 50;  
10     Student studentList[SIZE];  
11     int index = 0;  
12     //.... etc  
13 }
```

Program uses an array of structs to process student data:

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4     //type definition
5     struct Student {
6         string studentNum;
7         string firstName;
8         string surname;
9         int marks;
10    };
11    //function prototypes
12    Student getStudentData();
13    void showStudentData(Student oneStudent);
14    int averageMark(const Student StudentList[], int size);
```

Program uses an array of structs to process student data (2):

```
1  int main() {
2      const int SIZE = 4;
3      Student studentList[SIZE];
4      int classAverage;
5      //1. read student data into the studentList array
6      for (int index = 0; index < SIZE; index++)
7          studentList[index] = getStudentData();
8      //2. display student details
9      cout << "Student-details-are:-" << endl;
10     for (int index = 0; index < SIZE; index++)
11         showStudentData(studentList[index]);
12     //3.compute and display the average mark
13     classAverage = averageMark(studentList, SIZE);
14     cout << "Class-average-mark=-" << classAverage << endl;
15     return 0;
16 }
```

Program uses an array of structs to process student data (3):

```
1 Student getStudentData() {  
2     Student oneStudent;  
3     cout << endl;  
4     cout << "Enter student details:-" << endl;  
5     cout << "Student-number:-";  
6     cin >> oneStudent.studentNum;  
7     cout << "First-name:-";  
8     cin >> oneStudent.firstName;  
9     cout << "Surname:-";  
10    cin >> oneStudent.surname;  
11    cout << "Marks%:-";  
12    cin >> oneStudent.marks;  
13  
14    return oneStudent;  
15 }
```

Complete the code for the function below. You have 2 minutes.

```
1 void showStudentData(Student oneStudent)
2 {
3     cout << endl;
4     cout << "Student-number: ---"
5         << "-----"
6
7     cout << "First-name: ---"
8         << "-----"
9
10    cout << "Surname: ---"
11        << "-----"
12
13    cout << "Marks%: ---"
14        << "-----"
15 }
```

Class Solution:

```
1 void showStudentData(Student oneStudent) {  
2     cout << endl;  
3     cout << "Student-number:-" <<  
4     oneStudent.studentNum << endl;  
5  
6     cout << "First-name:-" <<  
7     oneStudent.firstName << endl;  
8  
9     cout << "Surname:-" <<  
10    oneStudent.surname << endl;  
11  
12    cout << "Marks%:-" <<  
13    oneStudent.marks << endl;  
14 }
```


Class Exercise (2)

Use the provided algorithm to write the code for the function provided below:

Algorithm:

1. `sum = 0, index = 0`
2. **while** `index < size`
 - 2.1 `sum += studentList[index].marks`
 - 2.2 `index++`
3. **return** `sum / size`

```
int averageMark(const Student studentList[], int size)
{
    //write your implementation here
}
```

Class Exercise Solution (2)

```
int averageMark(const Student studentList[], int size) {  
    int sum = 0, index = 0;  
    //1. sum the marks  
    while (index < size) {  
        sum += studentList[index].marks;  
        index++;  
    }  
    //2. compute average  
    average = sum / size;  
    return average;  
}
```

- A structure variable has an address
- A pointer to a structure is a variable that can hold the address of a structure:

```
Student aStudent;  
Student *stPtr;  
stPtr = &aStudent;
```

- Structure pointer can be a function parameter.

Pointer Implementation

```
1 struct Student {  
2     string studentNum;  
3     string firstName;  
4     string surname;  
5     int marks;  
6 };  
7  
8 int main() {  
9     Student aStudent;  
10    Student * stPtr;  
11  
12    stPtr = & aStudent;  
13    //..... etc  
14 }
```

Pointer Implementation

```
1  struct Student {
2      string studentNum;
3      string firstName;
4      string surname;
5      int marks;
6  };
7
8  int main() {
9      Student aStudent;
10     Student * stPtr;
11
12     stPtr = & aStudent;
13     //Must use ( ) to dereference pointer variable
14     cin >> (*stPtr).studentNum;
15     //Can use structure pointer operator -> to eliminate ( )
16     //This is a clearer notation
17     cin >> stPtr->studentNum; //most commonly used notation
18 }
```

Example of a struct Declaration:

Using dot notation	Using Pointer notation
<pre> void showStudentData (Student aStudent) {cout << "Student-number:-"<< aStudent.studentNum <<endl; cout << "First-name:-" << aStudent.firstName << endl; cout << "Surname:-" << aStudent.surname <<endl; cout << Marks : " --<<aStudent.marks; } </pre>	<pre> void showStudentData(Student * aStudent) { cout << "Student-number:-" << aStudent->studentNum << endl; cout << "First-name:-" << aStudent->firstName << endl; cout << "Surname:-" << aStudent->surname << endl; cout << Marks : "-----<< ----aStudent->-marks-<< ----endl; } </pre>

Program uses an array of pointers to structs to process student data:

```
1  struct Student {
2      string studentNum;
3      string firstName;
4      string surname;
5      int marks;
6  };
7  //function prototypes
8  void getStudentData(Student * aStudent);
9
10 void showStudentData(Student * aStudent);
11
12 int averageMark(Student * studentList[], int size);
```

Program uses an array of pointers to structs to process student data (2):

```
1  int main() {
2      const int SIZE = 4;
3      Student * studentList[SIZE];
4      Student * aStudent = NULL;
5      int index;
6      int classAverage;
7
8      //1. create the structs and store their addresses in the array
9      for (index = 0; index < SIZE; index++)
10         studentList[index] = new Student;
11     // ... continued
12     //2. read student data into the student list
13     for (int index = 0; index < SIZE; index++)
14         getStudentData(studentList[index]);
15     // NOTE: studentList[index] is a pointer
16
17     //3. display student details
18     cout << "Student-details:-" << endl;
19     for (int index = 0; index < SIZE; index++)
20         showStudentData(studentList[index]);
21 }
```


Program uses an array of pointers to structs to process student data (3):

```
1  int main() {
2  //4. compute and display the average mark
3  classAverage = averageMark(studentList, SIZE);
4
5  cout << endl << "Class-average-mark==" <<
6   classAverage << endl;
7
8  //5. de-allocate memory for the structs
9
10 for (int index = 0; index < SIZE; index++)
11     delete studentList[index];
12 return 0;
13 }
```

Program uses an array of pointers to structs to process student data (4):

```
1  void getStudentData(Student * aStudent) {  
2      cout << endl;  
3      cout << "Enter student details:-" << endl;  
4  
5      cout << "Student-number:-";  
6      cin >> aStudent->studentNum;  
7  
8      cout << "First-name:-";  
9      cin >> aStudent->firstName;  
10  
11     cout << "Surname:-";  
12     cin >> aStudent->surname;  
13  
14     cout << "Marks%:-";  
15     cin >> aStudent->marks;  
16  
17 }
```

Program uses an array of pointers to structs to process student data (5):

```
1 void showStudentData(Student * aStudent) {  
2     cout << endl;  
3     cout << "-Student-details:-" << endl;  
4  
5     cout << "Student-number:" << aStudent->studentNum <<endl;  
6  
7     cout << "First-name:" << aStudent->firstName <<endl;  
8  
9     cout << "Surname:" << aStudent->surname <<endl;  
10  
11     cout << "Marks%:" << aStudent->marks << endl;  
12 }
```

Program uses an array of pointers to structs to process student data (6):

```
1  int averageMark(Student * studentList[], int size) {  
2      Student * aStudent;  
3      int sum = 0;  
4      int index = 0;  
5      while (index < size) {  
6          aStudent = studentList[index];  
7          sum += aStudent -> marks;  
8          index++;  
9      }  
10     average = sum / size;  
11     return average;  
12 }
```

- Creating a dynamic array of structs in C++ involves using pointers and dynamic memory allocation with the **new** and **delete** operators.
- You need to perform the following:
 - 1 **Define the Struct:** First, define the struct you want to use.
 - 2 **Declare a Pointer to the Struct:** This will hold the base address of the dynamic array.
 - 3 **Allocate Memory Dynamically:** Use the **new** operator to allocate memory for the array.
 - 4 **Access and Modify the Elements:** Use the pointer to access and modify elements of the array.
 - 5 **Release the Memory:** Use the **delete[]** operator to free the allocated memory when done.

Program that shows a Dynamic Array of Structs (1):

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 // Define the Student structure
6 struct Student {
7     string studentNum;
8     string firstName;
9     string surname;
10    int marks;
11 };
12
13 int main() {
14    int size;
15    cout << "Enter the number of students: -";
16    cin >> size;
17    // Allocate memory for an array of Students dynamically
18    Student* studentList = new Student[size];
```

Program that shows a Dynamic Array of Structs (2):

```
1 // Input student details
2 for (int i = 0; i < size; ++i) {
3     cout << "Enter student number for student-" << i + 1 << ":-";
4     cin >> studentList[i].studentNum;
5     cout << "Enter first name for student-" << i + 1 << ":-";
6     cin >> studentList[i].firstName;
7     cout << "Enter surname for student-" << i + 1 << ":-";
8     cin >> studentList[i].surname;
9     cout << "Enter marks for student-" << i + 1 << ":-";
10    cin >> studentList[i].marks;
11 }
```

Program that shows a Dynamic Array of Structs (3):

```
1 // Display student details
2 for (int i = 0; i < size; ++i) {
3     cout << "Student-" << i + 1 << ":-" << endl;
4     cout << "--Student-Number:-" << studentList[i].studentNum << endl;
5     cout << "--First-Name:-" << studentList[i].firstName << endl;
6     cout << "--Surname:-" << studentList[i].surname << endl;
7     cout << "--Marks:-" << studentList[i].marks << endl;
8 }
9
10 // Free the dynamically allocated memory
11 delete[] studentList;
12
13 return 0;
14 }
```


Dynamic Array Exercise from L31

Header File (matrix.h):

```
1 #ifndef MATRIX_H
2 #define MATRIX_H
3 //Defining struct Matrix
4 struct Matrix {
5     int rows;
6     int cols;
7     float** data;
8 };
9 Matrix allocateMatrix(int rows, int cols);
10 void deallocateMatrix(Matrix matrix);
11 void readMatrix(Matrix& matrix);
12 void printMatrix(const Matrix& matrix);
13 Matrix addMatrices(const Matrix& a, const Matrix& b);
14 Matrix multiplyMatrices(const Matrix& a, const Matrix& b);
15 #endif
```

Implementation File (matrix.cpp):

```

1  #include <iostream>
2  #include "matrix.h"
3  using namespace std;
4
5  Matrix multiplyMatrices(const Matrix& a, const Matrix& b) {
6  if (a.cols != b.rows) { //check if matrix dimensions are similar
7      cout << "Error:-Incompatible-matrix-dimensions" << endl;
8      return allocateMatrix(0, 0); // Return an empty matrix
9  }
10
11  Matrix result = allocateMatrix(a.rows, b.cols);
12
13  for (int i = 0; i < result.rows; ++i) {
14      for (int j = 0; j < result.cols; ++j) {
15          result.data[i][j] = 0.0f; // Initialize to zero
16          for (int k = 0; k < a.cols; ++k) {
17              result.data[i][j] += a.data[i][k] * b.data[k][j];
18          }
19      }
20  }
21  return result;
22  }

```

Main Program (main.cpp) (1):

```
1  #include <iostream>
2  #include "matrix.h"
3
4  using namespace std;
5
6  int main() {
7      int rowsA, colsA, rowsB, colsB;
8
9      cout << "Enter the num-of-rows-and-columns-for-matrix-A: -";
10     cin >> rowsA >> colsA;
11
12     Matrix matrixA = allocateMatrix(rowsA, colsA);
13     readMatrix(matrixA);
14
15     cout << "Enter the num-of-rows-and-columns-for-matrix-B: -";
16     cin >> rowsB >> colsB;
17
18     Matrix matrixB = allocateMatrix(rowsB, colsB);
19     readMatrix(matrixB);
```

Main Program (main.cpp) (2):

```
1
2  if (colsA == rowsB) {
3      Matrix productMatrix = multiplyMatrices(matrixA, matrixB);
4      cout << "Product of matrices A and B:" << endl;
5      printMatrix(productMatrix);
6      deallocateMatrix(productMatrix);
7  } else {
8      cout << "Matrices are of incompatible dimensions." << endl;
9  }
10
11  deallocateMatrix(matrixA);
12  deallocateMatrix(matrixB);
13
14  return 0;
15 }
```

Things to note:

- Here we are assuming that you have already done implementations of the other functions in the header file (`matrix.h`)
- **Matrix Struct:** The Matrix struct holds the number of rows, columns, and the 2D array data for the matrix.
- **Header File (`matrix.h`):** Holds the library of matrix functions.
- **Implementation File (`matrix.cpp`):** Implements the functions for allocating, deallocating, reading, printing, adding, and multiplying matrices.
- **Main Program (`main.cpp`):** Tests the matrix functions by reading two matrices from the user, performing addition and multiplication (if possible), and printing the results.